Towards Efficient Educational Chatbots: Benchmarking RAG Frameworks

Umar Ali Khan
Electronics Engineering
Aligarh Muslim University
Aligarh, India
samkhan705279@gmail.com

Fiza Khan
Electronics Engineering
Aligarh Muslim University
Aligarh, India
fiza1892003@gmail.com

Ekram Khan
Electronics Engineering
Aligarh Muslim University
Aligarh, India
ekhan.el@amu.ac.in

Mohd Areeb Hasnain
Electronics Engineering
Aligarh Muslim University
Aligarh, India
areeb.hasnain678@gmail.com

Athar Ali Moinuddin
Electronics Engineering
Aligarh Muslim University
Aligarh, India
aamoin@zhcet.ac.in

2025 International Conference on Emerging Technologies in Computing and Communication (ETCC) | 979-8-3315-2476-0/25/$31.00 ©2025 IEEE | DOI: 10.1109/ETCC65847.2025.11108323

***Abstract*—Large Language Models (LLMs) have proven immensely beneficial in education by capturing vast amounts of literature-based information, allowing them to generate context without relying on external sources. In this paper, we propose a generative AI-powered GATE question-answering framework (GATE stands for Graduate Aptitude Test in Engineering) that leverages LLMs to explain GATE solutions and support students in their exam preparation. We conducted extensive benchmarking to select the optimal embedding model and LLM, evaluating our framework based on criteria such as latency, faithfulness, and relevance, with additional validation through human evaluation. Our chatbot integrates state-of-the-art embedding models and LLMs to deliver accurate, context-aware responses. Through rigorous experimentation, we identified configurations that balance performance and computational efficiency, ensuring a reliable chatbot to serve students' needs. Additionally, we discuss the challenges faced in data processing and modelling and the implemented solutions. Our research investigates the implementation of Retrieval-Augmented Generation (RAG) for GATE Q/A explanation tasks, and our findings demonstrate significant improvements in retrieval accuracy and response quality. This research offers practical insights for developing effective AI-driven educational tools while highlighting areas for future enhancement in usability and scalability.**

Keywords—Retrieval Augmented Generation, Information systems, Generative AI, Evaluation

I. Introduction

Large Language Models (LLMs) have proven immensely beneficial in education by capturing vast amounts of literature-based information, enabling them to generate context without needing external sources. Since the introduction of BERT [1], these models have been utilised across various tasks, including question answering and language inference, without necessitating substantial task-specific architectural alterations. As these models have scaled, they have increasingly packed more knowledge into their parameters [2]. However, while LLMs serve as useful repositories of information, they also come with notable limitations. First, they can only provide details based on the data available at the time of training, meaning they cannot address events that occur afterwards. Second, even the most advanced models often struggle to retain information that appears infrequently in their training data. For educational applications, it is essential to keep these models up-to-date. To reduce hallucinations [18] and enhance personalisation [9,17], current approaches rely on knowledge grounding [13,14] through retrieval-augmented generation (RAG) [4]. RAG can retrieve knowledge from external databases by integrating parametric and non-parametric memory, leading to diminished hallucinations and enhanced interpretability in tasks such as question answering and summarisation. Recent studies have shown that simply appending retrieved documents to a standard language model's input can yield effective results [8]. This indicates that retrieval-augmented techniques can be integrated with LLMs even when those models are accessed solely via APIs. Testing and evaluating RAG systems is crucial because it verifies that combining retrieval-based strategies with generative models truly enhances practical performance [10,12].

In this paper, we propose a GATE question-answering framework (GATE stands for Graduate Aptitude Test in Engineering) that leverages state-of-the-art LLMs to explain GATE solutions and support students in their exam preparation. We conducted extensive benchmarking to select the optimal embedding model and LLM, and we evaluated our framework based on several criteria, including latency, faithfulness, answer relevance, Exact Match (EM), and F1 score. Moreover, our evaluation strategy combines both LM probability estimates and human annotations, with the results averaged to ensure the most accurate benchmarks.

II. Background And Related Work

Generative AI-powered educational chatbots offer immense potential to provide virtual support to researchers, students, and educators. Students, in particular, benefit in three key areas: skill development, personalised learning experiences through coach-like modes, and assistance with study and homework tasks. In the past, obtaining the necessary information required extensive web searches, but now it can be accessed within a single burst of a prompt. While these chatbots save considerable time and enhance learning efficiency, they still face several limitations that need further improvement. Educational chatbot systems typically follow a two-stage pipeline: first, they retrieve related solutions by finding relevant chunks from a knowledge base; second, they use machine comprehension to generate an answer from the selected documents. Recently, retrieval-augmented generation (RAG) architectures have attracted significant

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 02,2026 at 15:57:37 UTC from IEEE Xplore. Restrictions apply.

TABLE I. EXAMPLES OF GROUND TRUTH RETRIEVED SOLUTIONS AND LLM EXPLANATIONS.

LLM	Retrieved Solution	LLM Explanation
Phi-3	(c) Here we will check tones. Mr. X speaks neither Japanese nor Chinese.	The solution is saying that Mr. X doesn't speak Japanese OR Chinese, which is the meaning of "neither" and "nor".
Phi-3	(d) Employing Insecticides to get rid of mosquitoes can have unintended outcomes; yet, the potential adverse effects of utilizing artificially created mosquitoes remain ambiguous.	The solution can be explained as using chemicals to kill mosquitoes may have undesired consequences while using genetically modified mosquito's consequences are still unknown.
llama3:8b-instruct-q2_K	(a, c) At equilibrium n=p for intrinsic semiconductor Collector region is generally lightly doped then base region in B J T. Hence option B is wrong. By increasing temperature above 300K, mobility of electrons decreases hence option (d) is also wrong	At equilibrium, the density of electrons and holes are equal. The correct statement is that the total current is spatially constant in a two-terminal electronic device in dark under steady-state conditions (c), but option (B) is incorrect because Collector region is generally lightly doped than Base region in a BJT. Also, mobility of electrons decreases with temperature above 300K in Silicon (d).
llama3:8b-instruct-q2_K	(a) \n[image1] LSB 0, 1, 0,1 For JK flip-flop (FF), 00 will not change the state. So, output frequency mathrm{f}_{0}/ 2 because two time change of state and duty cycle \n[image2] Duty cycle =50	The JK flip-flop (FF) changes state twice, resulting in an output frequency of f0/2 and a duty cycle of 50%.

[a.] The LLMs generally take into account the semantics of the retrieved solutions while explaining the solutions.

attention due to their explainability, scalability, and adaptability. In a RAG system, a query is initially encoded into a compact representation, subsequently relevant passages are extracted from an indexed knowledge storage, and ultimately, these passages are input into the generator. The loss function was designed specifically to concurrently optimise both the generator and the question encoder. Recent studies have shown that RAG models tend to produce more accurate responses because they rely on real documents, thereby mitigating the issue of generative models hallucinating information.

Evaluating LLM systems is equally important to ensure both faithfulness and context relevance. Recent work [7] has employed LLMs to automatically evaluate response quality beyond factuality, using prompts that specify the quality aspect, such as faithfulness or context relevance, and thereafter evaluating the responses according to the mean likelihood of the produced tokens from an autoregressive LM. Although prompt-based evaluation using a smaller fine-tuned LM (e.g., BART) was previously explored [6], it did not yield a clear advantage. Another approach involves asking ChatGPT to evaluate answer quality by assigning a score between 0 and 100 or a corresponding rating [20]. While this method can be effective, its success heavily depends on the prompt design. However, certain researchers employ LLMs to identify the optimal response from a pool of candidates.[3]. Although this technique is useful for comparing the performance of different LLMs, the sequence of answer presentation can affect the result.

In this paper, we have integrated several existing evaluation techniques and averaged the results using a nested RAG approach (detailed in Section V) to assess the generated explanations. Our work aims to explore the application of RAG for GATE Q/A explanation tasks. This work represents the first exploration into the application of RAG for explaining GATE questions and solutions.

III. FRAMEWORK ARCHITECTURE

We divided the development of our framework into two main components: frontend and backend. On the backend, we began by collecting GATE question solution data from various sources and parsing it (detailed in Section IV). After applying several chunking strategies to the raw data, we converted it into vector embeddings using our chosen embedding model and stored these embeddings along with their associated data in our Weaviate vector database. We also added metadata tags to link images to their corresponding data, with the images stored separately under the same tags.

Once the database was set up, we moved on to benchmarking various LLMs (details in Section IV). After selecting the appropriate model, we used Ollama to run both the LLM and the embedding model locally, which laid a solid foundation for our backend. Figure 1 shows the architecture of our framework, outlining the end-to-end flow from data preprocessing and embedding generation to model inference and frontend interaction.

For the frontend, we implemented JWT-based authentication for secure user login and used MongoDB to store user data, feedback, and chat history. We also integrated an AI note-taking feature that leverages a quantized version of Llama3.8 B-instruct to summarize conversations into concise insights. After completing both frontend and backend development, we containerized the entire application using Docker for seamless deployment.

With all components connected, the framework works as follows: an authenticated user selects a question from a searchable dropdown list on the frontend. The backend retrieves the exact solution along with the appropriate citations and embedded images, and displays it along with a follow-up prompt for input. When the user submits a related query, the Q/A pair, along with any parsed image data, is fed to the LLM as reference material, and an explanation is generated according to the prompt template. Table I shows examples of ground truth retrieved solutions. Finally, the user can summarize the conversation, take notes, and even initiate multiple sessions.

IV. METHODOLOGY

To effectively address the challenge of GATE question-answer explanation, we structured our framework development into two key phases: (1) Data Processing, Evaluation, and Modeling, and (2) Full Stack Development, System Design, and Deployment.

A. Data Processing, Evaluation, and Modelling

For the data processing, evaluation, and modelling phase, we started by identifying reliable sources for GATE question-answer explanations, including MadeEasy GATE solution [15] and Byju's GATE solutions [16].

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 02,2026 at 15:57:37 UTC from IEEE Xplore. Restrictions apply.

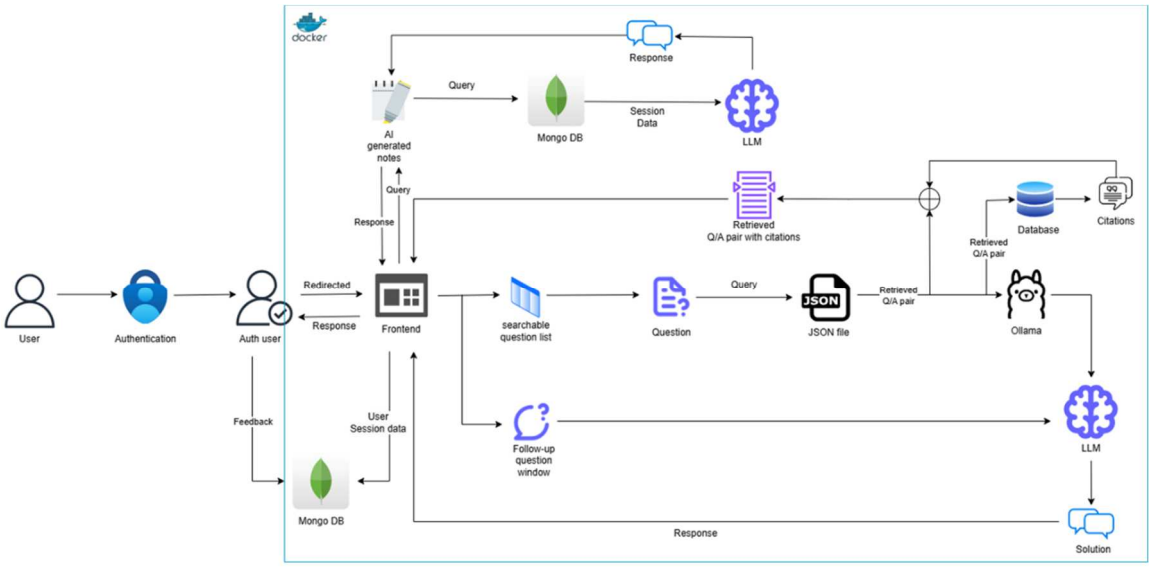


Fig. 1. System Overview of RAG Framework Architecture

After selecting the relevant GATE question-solution PDFs, we explored different data extraction techniques and assessed various tools such as Mathpix (paid, with limited free extractions) and open-source alternatives like Nought OCR [5], Tesseract OCR, LaTeX OCR, and LLaMA Parse for extracting textual content. To determine the most effective extraction method, we evaluated each tool based on key data quality metrics, including coverage, consistency, latency, and relevance, supplemented by human evaluation.

During the initial phase, we encountered significant challenges in extracting complex mathematical equations. Simple Python PDF libraries such as pypdf2 and pymupdf were inadequate for this task. As an alternative, we experimented with Tesseract OCR and Nought OCR, which showed improved accuracy in extracting mathematical expressions. However, for highly intricate equations, Mathpix proved to be the most reliable solution. Since our dataset included both text and image-based question-answer pairs, we also explored multimodal models like LLaVA to extract information from images. We also investigated image parsing techniques, using the Unstructured library to extract data from images, with the goal of providing question and solution image context to the LLM. This enabled us to incorporate visual data into the LLM-driven retrieval chain, enhancing LLM response.

Once the data was extracted, we focused on cleaning and structuring it for efficient chunking, which was a crucial step before integrating it into the retrieval-augmented framework. The next step was to clean and prepare the extracted data for chunking. We explored various chunking strategies, including semantic chunking, to effectively segment the data into Q&A pairs. We then decided to store the data in JSON format to ensure efficient ingestion into a vector database. Additionally, we generated metadata tags to categorize questions by type, year, and domain, ensuring a structured and organized dataset. Once data extraction and preparation were complete, we shifted our focus to data modelling.

For the modelling phase, we began by establishing benchmarks to guide our selection of the most suitable embedding and language models. For embedding models, we considered the Massive Text Embedding Benchmark. In the case of LLMs, we evaluated several key benchmarks, including:

- Chatbot Arena: A crowdsourced, randomized battle platform that aggregates over 70,000 user votes to compute ratings.
- MT-Bench: A collection of challenging multi-turn questions, where model responses are graded by GPT-4.
- MMLU (5-shot): A test intended to assess a model's multitask accuracy over 57 distinct tasks.

Additionally, we considered other established benchmarks such as GLUE, SUPERGLUE, HELM, and BIG-Bench to further refine our model selection and ensure optimal performance. After a careful review of these benchmarks and relevant leaderboards, we proceeded to the LLM selection process. The primary task was to retrieve math-related documents relevant to a query and provide a follow-up response based on the retrieved content. Key evaluation parameters included model size (in millions of parameters), memory usage (in GB, fp32), and performance on math-related tasks.

To inform our decision, we referred to the following leaderboards:

- LMSYS Chatbot Arena Leaderboard
- Open LLM Leaderboard

For large models (approximately 7 billion parameters), we compared:

- Mistral-7B
- Gemma-7B
- Llama-3-8B
- phi 3.8B
- Qwen1.5-7B

Based on these comparisons—and after initially considering phi-3.8B—we ultimately chose llama3:8b-instruct-q2_K, a quantized version of Llama3 that

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 02,2026 at 15:57:37 UTC from IEEE Xplore. Restrictions apply.

demonstrated exceptional performance within its parameter range. For the embedding model, we selected BGE Small, guided by the Massive Text Embedding Benchmark. Here, the focus was on retrieval, with key parameters including model size and memory usage. We consulted the MTEB Leaderboard to identify models that met our use case, especially those performing well on datasets such as CQADupstackRetrieval and 2HotpotQA.

Our objective was to achieve a robust latency-performance trade-off suitable for production. We began with a small, lightweight model to establish a baseline, with the option to upgrade to a larger, more powerful model later if needed.

Potential embedding models we considered included:

- nomic-ai/nomic-embed-text-v1.5
- Snowflake/snowflake-arctic-embed-m
- avsolatorio/GIST-Embedding-v0
- BAAI/bge-base-en-v1.5
- znbang/bge:small-en-v1.5-f32

Despite surpassing OpenAI's text-embedding-ada-002 and text-embedding-3-small models on both short and long context challenges, the nomic-embed-text model distinguished itself as a large-context text encoder, as noted in the Nomic Blog. Our human evaluations led us to choose BGE Small because of its outstanding performance on our mathematical dataset.

To ensure our model choices were optimal, we conducted latency tests to assess real-time performance and established specific data quality evaluation metrics to guarantee that the processed and retrieved data met the necessary standards for accuracy and reliability.

Once model selection and testing were finalized, we proceeded to populate our vector database. Initially, we employed CromaDB; however, we later transitioned to Weaviate due to its superior Dockerization support. We containerized the entire application to ensure a consistent and scalable deployment environment. Leveraging the processed data, we built a fully functional Retrieval-Augmented Generation (RAG) chatbot. During this phase, we encountered version-related issues with Weaviate, which we resolved by adhering closely to the official documentation.

In the subsequent version of our chatbot, we integrated image retrieval capabilities and added dual functionalities for both exact retrieval and follow-up question answering. This milestone enabled us to shift our focus toward enhancing user experience and refining the overall workflow. The new workflow allows users to search for questions from a provided list, displaying the exact solution based on their selection. Additionally, if users have further queries related to the chosen question, they can input a follow-up question in a prompt window, which is then addressed by the language model.

B. Full Stack Development, System Design, and Deployment

For this part, we began with extensive research into the best technologies for the frontend, backend, and databases, all aimed at creating an intuitive user experience. For the initial version of the Exam Chatbot Assistant, we pinpointed core features such as user authentication, chat history storage, and a feedback mechanism for chatbot responses. The user interface was designed in Figma to ensure both simplicity and functionality. This design included a login/signup page and a question-answer interface with a sidebar for filtering questions by exam type and year, thereby enhancing navigation and usability. We selected Next.js for the frontend due to its server-side rendering capabilities and seamless integration with the backend. JWT-based authentication provided secure login and efficient session management, while the responsive design catered to users across multiple devices. On the backend, FastAPI powered our APIs for storing chat history and handling feedback, chosen for its high performance and support for asynchronous operations. MongoDB was employed as our primary database because of its flexibility and scalability.

Integrating these technologies presented challenges, including JavaScript errors, API issues, and difficulties with Weaviate integration. Overcoming these obstacles required extensive debugging, iterative refinements, and thorough code reviews. Docker played a crucial role in streamlining the deployment process and enhancing team collaboration. Through persistent effort, we delivered a functional version 0.1 that met our initial objectives and established a robust foundation for future improvements. In the next iteration, we enhanced the user experience further by adding features such as a searchable dropdown for filtering questions by exam type and year, which made navigation quicker and more intuitive. We also implemented session management to allow users to save and revisit conversations and introduced a note-taking feature that leverages an AI model (Llama3:8 B-instruct-q2_K) to summarize conversations into concise insights. To support these enhancements, session data, notes, and Q&A interactions were stored in MongoDB. Additionally, data previously stored in JSON files was migrated to MongoDB, improving both security and accessibility.

V. Evaluation And Results

A. Evaluation Strategies

We deployed our framework on a local machine. When a user submits a question, the system first retrieves the relevant context—essentially, the exact solution—and then uses that information to generate a follow-up response addressing the user's query regarding the solution. The quality of the responses is assessed based on three key criteria.

Faithfulness: This criterion ensures that the answers are firmly grounded in the provided context. By using the exact solution as a reliable reference, we minimize hallucinations and ensure that the retrieved context substantiates the explanation generated by the LLM.

Answer Relevance: This aspect verifies that the generated explanation directly addresses the follow-up prompt or query related to the original solution. To assess answer relevance, we employed techniques that use LLMs as evaluators. For our initial setup, we constructed LLM chains using Langgraph, and all explanations were evaluated using Llama 3.8B, available through Ollama. Table II summarizes the faithfulness and answer relevance scores achieved by different LLMs.

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 02,2026 at 15:57:37 UTC from IEEE Xplore. Restrictions apply.

TABLE II. USING THE GATE QUESTION ANSWER DATASET, COMPARE FAITHFULNESS AND ANSWER RELEVANCE, INCORPORATING FEEDBACK FROM HUMAN ANNOTATORS.

LLM	Faith.	Ans. Rel.
llama3:8b-instruct-q2_K	0.94	0.83
Phi-3	0.87	0.78
GPT-3.5-turbo	0.56	0.49

Automated Evaluation Setup: To measure both faithfulness and answer relevance in a fully automated manner, we implemented the following procedures:

Our first evaluation strategy asses the explanation on two criteria faithfulness and answer relevance [11].

- For **faithfulness**, if all of the assertions in a response, q, can be logically inferred from its grounded context, c(q), then we assume that q is faithful to c(q). The final faithfulness score, F, is calculated as follows:

$$F = \frac{|V|}{|S|}$$

where |V| is the number of statements supported by the LLM and |S| is the total number of statements. We employ an LLM to determine whether the explanation is supported by the context.

- For **answer relevance**, we generate n potential questions based on the generated explanation using LLMs and then calculate a similarity score between these generated questions and the original question.

Our final evaluation strategy (nested RAG) involved splitting our dataset into an 80-20 ratio. We manually annotated 20% of the data for both faithfulness and answer relevance. These annotations, along with the corresponding explanation, original question, and follow-up prompt, were then used as references for the LLM to evaluate the remaining 80% of the data. More precisely, we have used another RAG to cross-validate the results of another RAG system, and the final evaluation scores represent the average of both strategies. Furthermore, we used quantitative measures like Top-k retrieval accuracy, F1 score, and Exact Match (EM). The F1 score records word overlap regardless of order, whereas the EM score quantifies the precise word-level match between the expected and actual responses. Table III presents the average performance of various LLM and embedding model combinations based on Faithfulness, F1, and EM scores.

TABLE III. PERFORMANCE OF THE VARIOUS RAG MODELS THAT WE TESTED IN

LLM	Embedding Model	EM	F1
Phi-3	bge:small-en-v1.5-f32	0.83	24.81
Phi-3	nomic-embed-text:latest	0.62	23.12
llama3:8b-instruct-q2_K	bge:small-en-v1.5-f32	0.07	15.33
llama3:8b-instruct-q2_K	nomic-embed-text:latest	0.14	14.83

In addition, we performed several automated latency measurement tests to compare the performance of various LLM and embedding model combinations, ensuring that we identify the configuration that delivers the best overall performance.

TABLE IV. SHOWS END-TO-END LATENCY MEASUREMENT TEST AVERAGE TIME (IN SECONDS) LOGS.

LLM	Embedding Model	Solution Retrieval	Explanation Generation
Phi-3	bge:small-en-v1.5-f32	0.91	4.61
Phi-3	nomic-embed-text:latest	1.11	7.18
llama3:8b-instruct-q2_K	bge:small-en-v1.5-f32	0.97	9.35
llama3:8b-instruct-q2_K	nomic-embed-text:latest	1.38	11.01

B. Results

Our experimental results indicate that, in terms of latency, the LLM Phi-3 performs faster, while in terms of context relevance and faithfulness, the quantized version of Llama3.8B demonstrates a clear advantage. Table IV lists the solution retrieval and generation latencies for each tested configuration. Although Llama3.8B incurs a higher inference time, it generates significantly more context-aware responses, which is sufficiently convincing for us to choose the LLM Llama3.8B quantized version together with BGE:small-en-v1.5-f32 as the embedding model for the final version of our framework.

VI. CONCLUSION

In this paper, we proposed a novel application of RAG and assessed its performance in explaining GATE question solutions. We provided a comprehensive discussion of the framework pipeline to encourage future research. We presented several benchmarks and identified configurations that yield low latency and high context relevance, and discussed our approach to improving the overall user experience. Our findings underscore the importance of selecting the appropriate models to suit our task. Furthermore, we explored various evaluation strategies using LLMs as both judges and juries. Future work could involve incorporating agents using LangGraph in the retrieval process, which may significantly enhance the overall performance of the framework. We recommend for the exploration of additional strategies that may enhance the overall efficiency of RAG.

REFERENCES

[1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. “BERT: Pre-training of deep bidirectional transformers for language understanding”.

[2] Adam Roberts, Colin Raffel, and Noam Shazeer. 2020. “How much knowledge can you pack into the parameters of a language model?”.

[3] Peiyi Wang, Lei Li, Liang Chen, Dawei Zhu, Binghuai Lin, Yunbo Cao, Qi Liu, Tianyu Liu, and Zhifang Sui. 2023b. Large language models are not fair evaluators. CoRR, abs/2305.17926.

[4] Gupta, Shailja, Rajesh Ranjan, and Surya Narayan Singh. "A Comprehensive Survey of Retrieval-Augmented Generation (RAG): Evolution, Current Landscape, and Future Directions." arXiv preprint arXiv:2410.12837 (2024).

[5] Lukas Blecher, Guillem Cucurull, Thomas Scialom, Robert Stojnic, “Nougat: Neural Optical Understanding for Academic Documents,” [online]. Available: https://doi.org/10.48550/arXiv.2308.13418

[6] Weizhe Yuan, Graham Neubig, and Pengfei Liu. 2021. Bartscore: Evaluating generated text as text generation. In Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual, pages 27263–27277.

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 02,2026 at 15:57:37 UTC from IEEE Xplore. Restrictions apply.

[7] Jinlan Fu, See-Kiong Ng, Zhengbao Jiang, Pengfei Liu, “GPTScore: Evaluate as You Desire,“ [Online]. Available: https://arxiv.org/pdf/2302.04166

[8] Omar Khattab, Keshav Santhanam, Xiang Lisa Li, David Hall, Percy Liang, Christopher Potts, Matei Zaharia, “Demonstrate-Search-Predict: Composing retrieval and language models for knowledge-intensive NLP,” Dec. 2022. [Online]. Available: https://arxiv.org/abs/2212.14024

[9] Alireza Salemi, Sheshera Mysore, Michael Bendersky, and Hamed Zamani. 2023. LaMP: When Large Language Models Meet Personalization. arXiv:2304.11406 [cs.CL]

[10] Jithin James and Shahul Es. 2023. Ragas: Evaluation framework for your retrieval-augmented generation (rag) pipelines. https://github.com/explodinggradients/ ragas.

[11] Shahul Es, Jithin James, Luis Espinosa-Anke, Steven Schockaert, “RAGAS: Automated Evaluation of Retrieval Augmented Generation,” 2023, arXiv:2309.15217. [Online]

[12] Jon Saad-Falcon, Omar Khattab, Christopher Potts, and Matei Zaharia. 2023. ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems. arXiv:2311.09476 [cs.CL]

[13] PatrickLewis,EthanPerez,AleksandraPiktus,FabioPetroni,VladimirKarpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLPtasks.In Proceedings of the 34th International Conference on Neural Information Processing Systems (Vancouver, BC, Canada)

[14] Hamed Zamani and Michael Bendersky. 2024. Stochastic RAG: End-to-End Retrieval-Augmented Generation through Expected Utility Maximization. In Proceedings of the 47th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR ’24)

[15] https://www.madeeasy.in/gate-previous-year-solved-papers

[16] https://byjus.com/gate/gate-electronics-and-communication-engineering-ece-question-paper/

[17] Alireza Salemi, Surya Kallumadi, and Hamed Zamani. 2024. Optimization Meth ods for Personalizing Large Language Models through Retrieval Augmentation. In Proceedings of the 47th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR ’24).

[18] Garima Agrawal, Tharindu Kumarage, Zeyad Alghami, and Huan Liu. 2023. “Can Knowledge Graphs Reduce Hallucinations in LLMs?” : A Survey. arXiv:2311.07914 [cs.CL]

[19] U. A. Khan, E. Khan, F. Khan, and A. A. Moinuddin, Towards Efficient Educational Chatbots: Benchmarking RAG Frameworks, 2025 (Preprint) [Online]. Available: arxiv.org/pdf/2503.00781.

[20] Jiaan Wang, Yunlong Liang, Fandong Meng, Haoxi ang Shi, Zhixu Li, Jinan Xu, Jianfeng Qu, and Jie Zhou. 2023a. Is ChatGPT a good NLG evaluator? A preliminary study. CoRR, abs/2303.04048.

Authorized licensed use limited to: b-on: Instituto Politecnico do Porto. Downloaded on November 02,2026 at 15:57:37 UTC from IEEE Xplore. Restrictions apply.